

NPFL103: Information Retrieval (8)

Probabilistic Information Retrieval

Pavel Pecina

`pecina@ufal.mff.cuni.cz`

Institute of Formal and Applied Linguistics
Faculty of Mathematics and Physics
Charles University

Based on slides by Hinrich Schütze, University of Stuttgart.

Contents

Probabilistic approach to IR

Basic probability theory

Probability Ranking Principle

Binary Independence Model

Extensions

Probabilistic approach to IR

From relevance feedback to probabilistic IR

- ▶ In relevance feedback, the user marks documents as relevant and nonrelevant.
- ▶ Given some known relevant and nonrelevant documents, we compute weights for non-query terms that indicate how likely they will occur in relevant documents.
- ▶ We want to develop a **probabilistic approach for relevance feedback** and also a general **probabilistic model for IR**.

Probabilistic approach to retrieval

- ▶ Given a user information need (represented as a query) and a collection of documents (transformed into document representations), an IR system must determine *how well the documents satisfy the query*.
- ▶ An IR system has an **uncertain understanding** of the user query and makes an **uncertain guess** of whether a document satisfies the query.
- ▶ Probability theory provides a principled foundation for such **reasoning under uncertainty**.
- ▶ Probabilistic models exploit this foundation to estimate *how likely it is that a document is relevant to a query*.

Probabilistic IR models at a glance

- ▶ Classical probabilistic retrieval model
 1. Probability Ranking Principle (PRP)
 2. Binary Independence Model (BIM)
 3. BestMatch25 (BM25, Okapi)

- ▶ Language model approach to IR

Probabilistic model vs. other models

Boolean model:

- ▶ Probabilistic models support ranking and thus are better than the simple Boolean model.

Vector space model:

- ▶ The vector space model is also a formally defined model that supports ranking.
- ▶ Why would we want to look for an alternative to the vector space model?

Probabilistic vs. vector space model

- ▶ Vector space model: rank documents according to **similarity**.
- ▶ The notion of similarity does not translate directly into an assessment of “*is the document a good document to give to the user or not?*”
- ▶ The most similar document can be highly relevant or completely nonrelevant.
- ▶ Probability theory is arguably a cleaner formalization of what we really want an IR system to do: return **relevant** documents to the user.

Basic probability theory

Basic Probability Theory

- ▶ For events A and B :
 - ▶ **Joint probability** $P(A \cap B)$: both events occurring
 - ▶ **Conditional probability** $P(A|B)$: A occurring given B has occurred
- ▶ **Chain rule** gives relationship between joint/conditional probabilities:

$$P(AB) = P(A \cap B) = P(A|B)P(B) = P(B|A)P(A)$$

- ▶ Similarly for the complement of an event $P(\overline{A})$:

$$P(\overline{A}B) = P(B|\overline{A})P(\overline{A})$$

- ▶ **Partition rule**: if B can be divided into an exhaustive set of disjoint subcases, then $P(B)$ is the sum of the probabilities of the subcases. A special case of this rule gives:

$$P(B) = P(AB) + P(\overline{A}B)$$

Basic probability theory cont'd

- **Bayes' Rule** for inverting conditional probabilities:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{P(B|A)}{\sum_{X \in \{A, \bar{A}\}} P(B|X)P(X)} \cdot P(A)$$

- Can be thought of as a way of updating probabilities:
 - Start off with **prior probability** $P(A)$ (initial estimate of how likely event A is in the absence of any other information).
 - Derive a **posterior probability** $P(A|B)$ after having seen the evidence B , based on the likelihood of B occurring in the two cases that A does or does not hold.
- **Odds** of an event is a kind of multiplier for how probabilities change:

$$\text{Odds: } O(A) = \frac{P(A)}{P(\bar{A})} = \frac{P(A)}{1 - P(A)}$$

Probability Ranking Principle

The document ranking problem

- ▶ Ranked retrieval setup: given a collection of documents, the user issues a query, and an ordered list of documents is returned.
- ▶ Assume **binary relevance**: $R_{d,q}$ is a random dichotomous variable:
 $R_{d,q} = 1$ if document d is relevant w.r.t query q
 $R_{d,q} = 0$ otherwise.
- ▶ Often we write just R for $R_{d,q}$
- ▶ Probabilistic ranking orders documents decreasingly by their estimated probability of relevance w.r.t. query: $P(R = 1|d, q)$.
- ▶ Assume that the relevance of each document is independent of the relevance of other documents.

Probability Ranking Principle (PRP)

PRP in brief:

- ▶ If the retrieved documents (w.r.t a query) are ranked decreasingly on their probability of relevance, then the effectiveness of the system will be the best that is obtainable.

PRP in full:

- ▶ If [the IR] system's response to each [query] is a ranking of the documents [...] in order of decreasing probability of relevance to the [query], **where the probabilities are estimated as accurately as possible on the basis of whatever data have been made available to the system for this purpose**, the overall effectiveness of the system to its user will be the best that is obtainable on the basis of those data.

Binary Independence Model

Binary Independence Model (BIM)

Traditionally used with the PRP, with the following assumptions:

1. 'Binary' (equivalent to Boolean): documents and queries represented as binary term incidence vectors.
 - ▶ E.g., document d represented by vector $\vec{x} = (x_1, \dots, x_M)$, where $x_t = 1$ if term t occurs in d and $x_t = 0$ otherwise.
 - ▶ Different documents may have the same vector representation.
 - ▶ Similarly, we represent q by the incidence vector $\vec{q}$.
2. 'Independence': no association between terms (not true, but practically works)

Binary incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

Each document is represented as a **binary vector** $\in \{0, 1\}^{|V|}$.

Binary Independence Model (1)

To make a probabilistic retrieval strategy precise, need to estimate how terms in documents contribute to relevance:

- ▶ Find measurable statistics (term frequency, document frequency, document length) that affect judgments about document relevance.
- ▶ Combine these statistics to estimate the probability $P(R|d, q)$ of document relevance.
- ▶ Next: how exactly we can do this.

Binary Independence Model (2)

$P(R|d, q)$ is modeled using term incidence vectors as $P(R|\vec{x}, \vec{q})$:

$$P(R = 1|\vec{x}, \vec{q}) = \frac{P(\vec{x}|R = 1, \vec{q})P(R = 1|\vec{q})}{P(\vec{x}|\vec{q})}$$

$$P(R = 0|\vec{x}, \vec{q}) = \frac{P(\vec{x}|R = 0, \vec{q})P(R = 0|\vec{q})}{P(\vec{x}|\vec{q})}$$

- ▶ $P(\vec{x}|R = 1, \vec{q})$ and $P(\vec{x}|R = 0, \vec{q})$: probability that if a relevant or nonrelevant document is retrieved, then that document's representation is $\vec{x}$.
- ▶ Use statistics about the document collection to estimate these probabilities.

Binary Independence Model (3)

$P(R|d, q)$ is modeled using term incidence vectors as $P(R|\vec{x}, \vec{q})$:

$$P(R = 1|\vec{x}, \vec{q}) = \frac{P(\vec{x}|R = 1, \vec{q})P(R = 1|\vec{q})}{P(\vec{x}|\vec{q})}$$
$$P(R = 0|\vec{x}, \vec{q}) = \frac{P(\vec{x}|R = 0, \vec{q})P(R = 0|\vec{q})}{P(\vec{x}|\vec{q})}$$

- ▶ $P(R = 1|\vec{q})$ and $P(R = 0|\vec{q})$: prior probability of retrieving a relevant or nonrelevant document for a query $\vec{q}$.
- ▶ Estimate $P(R = 1|\vec{q})$ and $P(R = 0|\vec{q})$ from percentage of relevant documents in the collection.
- ▶ Since a document is either relevant or nonrelevant to a query, we must have that:
 $P(R = 1|\vec{x}, \vec{q}) + P(R = 0|\vec{x}, \vec{q}) = 1$

Deriving a ranking function for query terms (1)

- ▶ Given a query q , ranking documents by $P(R = 1|d, q)$ is modeled under BIM as ranking them by $P(R = 1|\vec{x}, \vec{q})$.
- ▶ Easier: rank documents by their odds of relevance (same ranking):

$$\begin{aligned} O(R|\vec{x}, \vec{q}) &= \frac{P(R = 1|\vec{x}, \vec{q})}{P(R = 0|\vec{x}, \vec{q})} = \frac{\frac{P(R=1|\vec{q})P(\vec{x}|R=1,\vec{q})}{P(\vec{x}|\vec{q})}}{\frac{P(R=0|\vec{q})P(\vec{x}|R=0,\vec{q})}{P(\vec{x}|\vec{q})}} = \\ &= \frac{P(R = 1|\vec{q})}{P(R = 0|\vec{q})} \cdot \frac{P(\vec{x}|R = 1, \vec{q})}{P(\vec{x}|R = 0, \vec{q})} \end{aligned}$$

- ▶ $\frac{P(R=1|\vec{q})}{P(R=0|\vec{q})} = O(R|\vec{q})$ is a constant for a given query $\rightarrow$ can be ignored.

Deriving a ranking function for query terms (2)

At this point we make the **Naive Bayes conditional independence assumption** that the presence/absence of a word in a document is independent of the presence/absence of any other word (given the query):

$$\frac{P(\vec{x}|R=1, \vec{q})}{P(\vec{x}|R=0, \vec{q})} = \prod_{t=1}^M \frac{P(x_t|R=1, \vec{q})}{P(x_t|R=0, \vec{q})}$$

So:

$$O(R|\vec{x}, \vec{q}) = O(R|\vec{q}) \cdot \prod_{t=1}^M \frac{P(x_t|R=1, \vec{q})}{P(x_t|R=0, \vec{q})}$$

Exercise

Naive Bayes conditional independence assumption: the presence or absence of a word in a document is independent of the presence or absence of any other word (given the query).

Why is this wrong? Good example?

PRP assumes that the relevance of each document is independent of the relevance of other documents.

Why is this wrong? Good example?

Deriving a ranking function for query terms (3)

- Since each x_t is either 0 or 1, we can separate the terms:

$$\begin{aligned} O(R|\vec{x}, \vec{q}) &= O(R|\vec{q}) \cdot \prod_{t=1}^M \frac{P(x_t|R=1, \vec{q})}{P(x_t|R=0, \vec{q})} = \\ &= O(R|\vec{q}) \cdot \prod_{t:x_t=1} \frac{P(x_t=1|R=1, \vec{q})}{P(x_t=1|R=0, \vec{q})} \cdot \prod_{t:x_t=0} \frac{P(x_t=0|R=1, \vec{q})}{P(x_t=0|R=0, \vec{q})} \end{aligned}$$

Deriving a ranking function for query terms (4)

- ▶ Let $p_t = P(x_t = 1 | R = 1, \vec{q})$ be the probability of a term appearing in relevant document.
- ▶ Let $u_t = P(x_t = 1 | R = 0, \vec{q})$ be the probability of a term appearing in a nonrelevant document.
- ▶ Can be displayed as a [contingency table](#):

document		relevant ($R = 1$)	nonrelevant ($R = 0$)
Term present	$x_t = 1$	p_t	u_t
Term absent	$x_t = 0$	$1 - p_t$	$1 - u_t$

Deriving a ranking function for query terms (5)

- ▶ Additional simplifying assumption: terms not occurring in the query are equally likely to occur in relevant and nonrelevant documents.

→ If $q_t = 0$, then $p_t = u_t$.

- ▶ We only consider terms in the products that appear in the query:

$$O(R|\vec{x}, \vec{q}) = O(R|\vec{q}) \cdot \prod_{t: x_t=q_t=1} \frac{p_t}{u_t} \cdot \prod_{t: x_t=0, q_t=1} \frac{1-p_t}{1-u_t}$$

- ▶ The left product is over query terms found in the document and the right product is over query terms not found in the document.

Deriving a ranking function for query terms (6)

- ▶ Including the query terms found in the document into the right product, but simultaneously dividing by them in the left product:

$$O(R|\vec{x}, \vec{q}) = O(R|\vec{q}) \cdot \prod_{t: x_t=q_t=1} \frac{p_t(1-u_t)}{u_t(1-p_t)} \cdot \prod_{t: q_t=1} \frac{1-p_t}{1-u_t}$$

- ▶ The left product is still over query terms found in the document, but the right product is now over all query terms, hence constant for a particular query and can be ignored.

→ The only quantity that needs to be estimated to rank documents w.r.t a query is the left product.

- ▶ We can equally rank documents by the logarithm of this term, since log is monotonic.
- ▶ Hence the **Retrieval Status Value** (RSV) in this model:

$$RSV_d = \log \prod_{t: x_t=q_t=1} \frac{p_t(1-u_t)}{u_t(1-p_t)} = \sum_{t: x_t=q_t=1} \log \frac{p_t(1-u_t)}{u_t(1-p_t)}$$

Deriving a ranking function for query terms (7)

Equivalent: rank documents using **log odds ratios** for the query terms c_t :

$$c_t = \log \frac{p_t(1 - u_t)}{u_t(1 - p_t)} = \log \frac{p_t}{(1 - p_t)} - \log \frac{u_t}{1 - u_t}$$

- ▶ The **odds ratio** is the ratio of two odds: (i) the odds of the term appearing if the document is relevant ($p_t/(1 - p_t)$), and (ii) the odds of the term appearing if the document is nonrelevant ($u_t/(1 - u_t)$)
- ▶ $c_t = 0$: term has equal odds of appearing in relevant and nonrel. docs.
- ▶ c_t positive: higher odds to appear in relevant documents.
- ▶ c_t negative: higher odds to appear in nonrelevant documents.

Term weight c_t in BIM

- ▶ $c_t = \log \frac{p_t}{(1 - p_t)} - \log \frac{u_t}{1 - u_t}$ functions as a term weight.
- ▶ Retrieval status value for document d : $RSV_d = \sum_{x_t=q_t=1} c_t$.
- ▶ So BIM and vector space model are identical on an operational level
... except that the term weights are different.
- ▶ In particular: we can use the same data structures (inverted index etc) for the two models.

How to compute probability estimates

For each term t in a query, estimate c_t in the whole collection using a contingency table of counts of documents in the collection, where df_t is the number of documents that contain t :

	documents	relevant	nonrelevant	Total
Term present	$x_t = 1$	s	$df_t - s$	df_t
Term absent	$x_t = 0$	$S - s$	$(N - df_t) - (S - s)$	$N - df_t$
	Total	S	$N - S$	N

$$p_t = s/S$$

$$u_t = (df_t - s)/(N - S)$$

$$c_t = K(N, df_t, S, s) = \log \frac{s/(S - s)}{(df_t - s)/((N - df_t) - (S - s))}$$

Avoiding zeros

- ▶ If any of the counts is a zero, then the term weight is not well-defined.
- ▶ Maximum likelihood estimates do not work for rare events.
- ▶ To avoid zeros: **add 0.5 to each count** (Expected Likelihood Estimation).
- ▶ For example, use $S - s + 0.5$ in formula for $S - s$.

Simplifying assumption

- ▶ Assuming that relevant documents are a very small percentage of the collection, approximate statistics for nonrelevant documents by statistics from the whole collection.
- ▶ Hence, u_t (the probability of term occurrence in nonrelevant documents for a query) is df_t/N and

$$\log[(1 - u_t)/u_t] = \log[(N - df_t)/df_t] \approx \log N/df_t$$

- ▶ This should look familiar to you ...
- ▶ The above approximation cannot easily be extended to relevant documents.

Probability estimates in relevance feedback

- ▶ Statistics of relevant documents (p_t) in relevance feedback can be estimated using maximum likelihood estimation or expected likelihood estimation (add 0.5).
 - ▶ Use the frequency of term occurrence in known relevant documents.
- ▶ This is the basis of probabilistic approaches to relevance feedback weighting in a feedback loop.

Probability estimates in ad-hoc retrieval

- ▶ Ad-hoc retrieval: no user-supplied relevance judgments available.
- ▶ In this case: assume that p_t is constant over all terms x_t in the query and that $p_t = 0.5$.
- ▶ Each term is equally likely to occur in a relevant document, and so the p_t and $(1 - p_t)$ factors cancel out in the expression for RSV .
- ▶ Weak estimate, but doesn't disagree violently with expectation that query terms appear in many but not all relevant documents.
- ▶ Combining this method with the earlier approximation for u_t , the document ranking is determined simply by which query terms occur in documents scaled by their idf weighting.

$$RSV_d = \sum_{t: x_t = q_t = 1} \log N/df_t$$

- ▶ For short documents (titles or abstracts) in one-pass retrieval situations, this estimate can be quite satisfactory.

Extensions

History and summary of assumptions

- ▶ Among the oldest formal models in IR:
 - ▶ Maron & Kuhns, 1960: Since an IR system cannot predict with certainty which document is relevant, we should deal with probabilities.
- ▶ Assumptions for getting reasonable approximations of the needed probabilities (in the BIM):
 1. Boolean representation of documents/queries/relevance
 2. Term independence
 3. Out-of-query terms do not affect retrieval
 4. Document relevance values are independent

How different are vector space model and BIM?

- ▶ They are not that different.
- ▶ In either case you build an information retrieval scheme in the exact same way.
- ▶ For probabilistic IR, at the end, you score queries not by cosine similarity and tf-idf in a vector space, but by a slightly different formula motivated by probability theory.
- ▶ Next: how to add term frequency and length normalization to the probabilistic model.

Okapi BM25: Overview

- ▶ Okapi BM25 is a probabilistic model that incorporates term frequency (i.e., it's nonbinary) and length normalization.
- ▶ BIM was originally designed for short catalog records of fairly consistent length, and it works reasonably in these contexts.
- ▶ For modern full-text search collections, a model should pay attention to term frequency and document length.
- ▶ BestMatch25 (a.k.a **BM25** or **Okapi**) is sensitive to these quantities.
- ▶ BM25 is one of the most widely used and robust retrieval models.

Okapi BM25: Starting point

The simplest score for document d is just **idf weighting** of the query terms present in the document:

$$RSV_d = \sum_{t \in q} \log \frac{N}{df_t}$$

Okapi BM25 basic weighting

Improve idf term by factoring in **term frequency** and **document length**.

$$RSV_d = \sum_{t \in q} \log \left[\frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b(L_d/L_{ave})) + tf_{td}}$$

- ▶ tf_{td} : term frequency in document d
- ▶ L_d : length of document d
- ▶ L_{ave} : average document length in the whole collection
- ▶ k_1 : tuning parameter controlling document term frequency scaling
- ▶ b : tuning parameter controlling scaling by document length

Exercise

Okapi BM25:

$$RSV_d = \sum_{t \in q} \log \left[\frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b(L_d/L_{ave})) + tf_{td}}$$

1. Interpret BM25 weighting formula for $k_1 = 0$
2. Interpret BM25 weighting formula for $k_1 = 1$ and $b = 0$
3. Interpret BM25 weighting formula for $k_1 \mapsto \infty$ and $b = 0$
4. Interpret BM25 weighting formula for $k_1 \mapsto \infty$ and $b = 1$

Okapi BM25 weighting for long queries

- ▶ For long queries, use similar weighting for query terms:

$$RSV_d = \sum_{t \in q} \left[\log \frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b(L_d/L_{ave})) + tf_{td}} \cdot \frac{(k_3 + 1)tf_{tq}}{k_3 + tf_{tq}}$$

- ▶ tf_{tq} : term frequency in the query q
- ▶ k_3 : tuning parameter controlling term frequency scaling of the query
- ▶ No length normalization of queries (because retrieval is being done with respect to a single fixed query)
- ▶ The above tuning parameters should ideally be set to optimize performance on a development test collection. In the absence of such optimization, experiments have shown reasonable values are to set k_1 and k_3 to a value between 1.2 and 2 and $b = 0.75$

Which ranking model should I use?

- ▶ I want something basic and simple → use vector space with tf-idf weighting.
- ▶ I want to use a state-of-the-art ranking model with excellent performance → use language models or BM25 with **tuned parameters**.
- ▶ In between: BM25 or language models with no or just one tuned parameter.

Outline

Probabilistic approach to IR

Basic probability theory

Probability Ranking Principle

Binary Independence Model

Extensions